

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

Факультет «Информатика и вычислительная техника»
Кафедра «Кибербезопасность информационных систем»

Лабораторная работа №6

по дисциплине: «Методы тестирования и отладки программного обеспечения»

На тему «Отладка программы с помощью pdb и графического отладчика PyCharm»

Выполнил обучающийся гр. ВКБ43
Ковалев Данил Петрович

Проверил:
Ковальчик Р.В.

Ростов-на-Дону
2026

1. Цель работы и исходная программа

Цель работы: освоить отладку Python-программы в консольном pdb и графическом отладчике PyCharm: установить точки останова, выполнить код по шагам, исследовать переменные и стек вызовов, исправить найденную ошибку и проверить результат.

По заданию допускается любой небольшой скрипт. Подготовлен учебный расчёт среднего арифметического с намеренно внесённой ошибкой границы цикла. Исходная версия сохранена в average_legacy.py, исправленная — в average.py; demo.py запускает обе версии на одинаковых данных.

Параметр	Значение
Входной список	[10, 20, 30, 40]
Контрольный расчёт	(10 + 20 + 30 + 40) / 4 = 25,00
Результат исходной версии	15,00 вместо 25,00
Среда	macOS; Python 3.14.3; PyCharm 2026.2.2

Таблица 1 — исходные данные и контрольный результат



Рисунок 1 — воспроизводимое расхождение исходной версии с эталоном

Расчёт не вызывает исключения, поэтому отсутствие traceback не означает правильность программы. Ключ --check выявляет неверный результат и возвращает код завершения 1. Проверяемые гипотезы: неверная граница цикла, ошибка накопления суммы, неверный делитель.

2. Отладка в pdb: остановка и выполнение по шагам

Запуск: .venv/bin/python -m pdb demo.py --version legacy. Команда b average_legacy.py:9 задаёт точку перед циклом, c продолжает выполнение до неё. p выводит значения, n выполняет следующую строку без входа во вложенные вызовы, display отслеживает изменение выражения [1].



Рисунок 2 — фрагмент реальной сессии pdb: p, display и два шага n

В начале total равен 0.0. После входа в цикл index равен 0; следующий шаг выполняет сложение и total становится 10.0. Выражение list(range(len(values) - 1)) уже показывает подозрительную границу: [0, 1, 2] при четырёх входных значениях.

Подсвечиваемая строка отладчика ещё не выполнена. Поэтому при остановке на total += values[index] значение total относится к состоянию до текущего сложения. Полные протоколы сохраняются локально в report_assets/logs/.

3. Отладка в pdb: локализация причины

Точка b average_legacy.py:11 останавливает программу перед возвратом среднего. После с проверены исходный список, последний индекс, накопленная сумма, длина списка и независимая сумма через sum(values).



Рисунок 3 — перед return получены index=2 и total=60.0

Проверка	Получено	Вывод
p values	[10, 20, 30, 40]	Входные данные не потеряны.
p (index, total, len(values))	(2, 60.0, 4)	Обработаны только первые три элемента.
p values[-1] / p sum(values)	40 / 100	Пропущено слагаемое 40.
p total / len(values)	15.0	Делитель 4 верен; неверен числитель.

Таблица 2 — наблюдения, различающие гипотезы

Гипотеза неверной границы цикла подтверждена. Накопление первых трёх значений выполнено верно: 10 + 20 + 30 = 60. Деление также корректно для полученной суммы: 60 / 4 = 15. Команда where дополнительно показала цепочку demo.py → main() → average(); стек включён в полный протокол сессии.

4. PyCharm: подготовка графической отладки

В PyCharm создана конфигурация Python «Lab 06 - legacy»: скрипт demo.py, параметры --version legacy --check, рабочий каталог лабораторной №6. Для выполнения выбран Python 3.14.3. Конфигурация запускается через Run → Debug.

В average_legacy.py установлены точки останова на строках 10 и 11. В панели Threads & Variables отображаются локальные значения; в Watches добавлены len(values), sum(values), list(range(len(values) - 1)) и values[-1]. Стек кадров расположен слева [2].

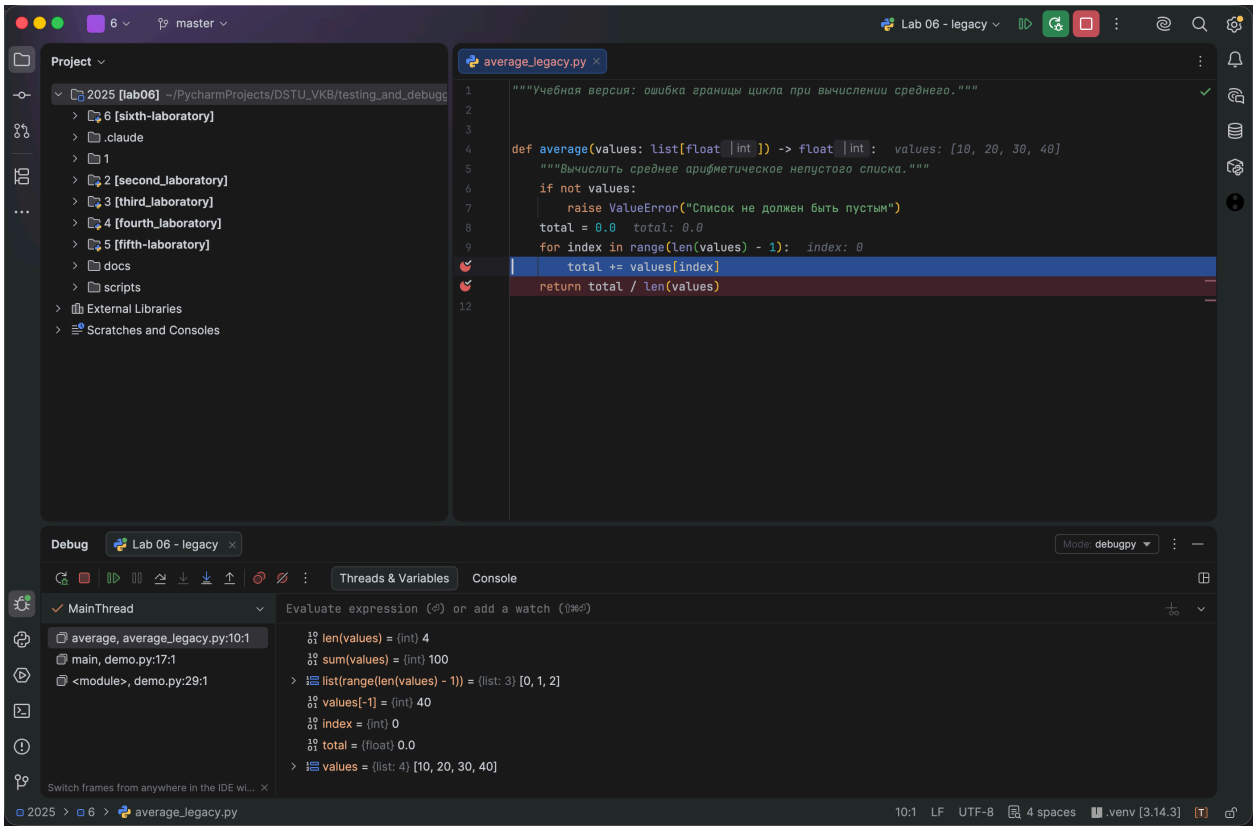


Рисунок 4 — реальная остановка PyCharm на первой итерации цикла

На первой итерации index=0 и total=0.0. Исходный список содержит четыре значения. Step Over выполняет сложение без входа во внутренние операции; Resume Program продолжает работу до следующей точки останова.

5. PyCharm: исследование состояния перед return

Выполнение продолжено до точки на строке 11. В редакторе выделена следующая исполняемая строка return total / len(values). Одновременно видны исходный код, стек, Watches и локальные переменные.

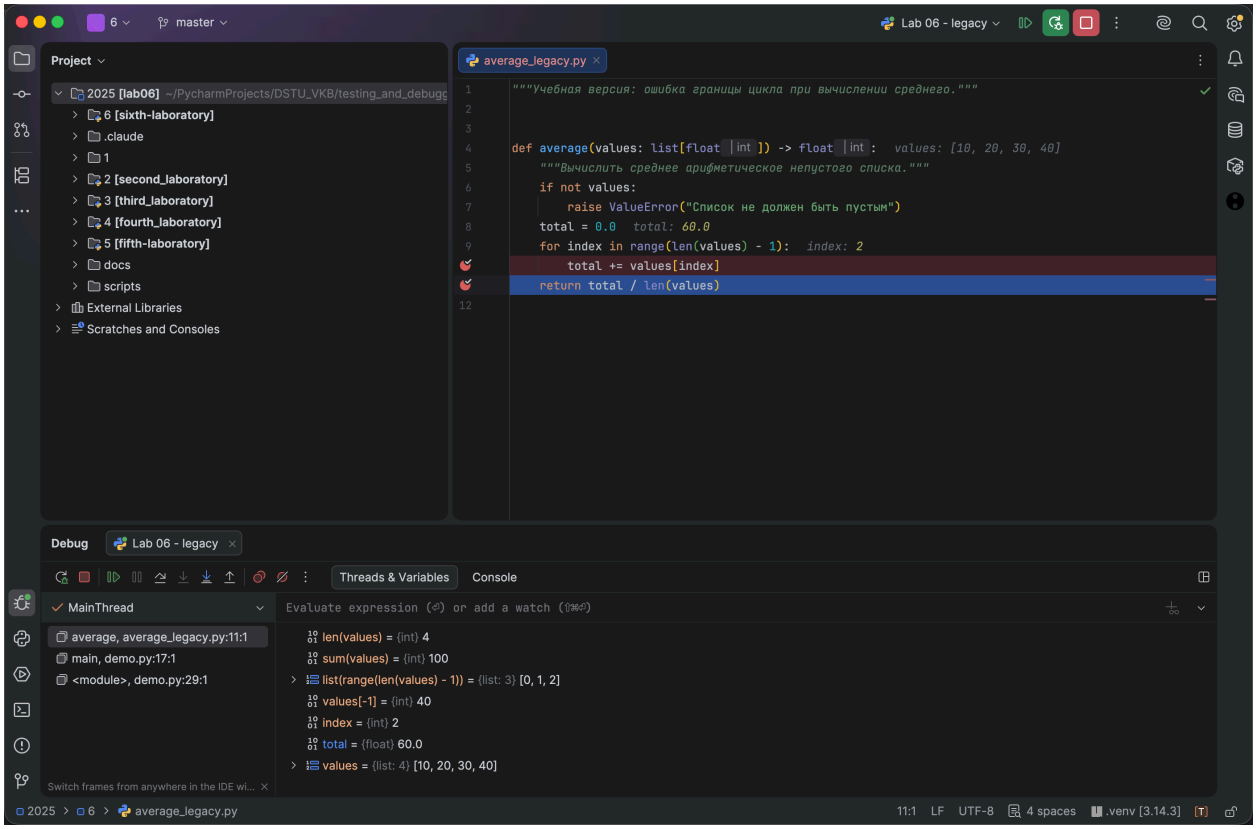


Рисунок 5 — PyCharm подтверждает пропуск последнего элемента

Объект наблюдения	Значение на снимке
index / total	2 / 60.0
len(values) / sum(values)	4 / 100
list(range(len(values) - 1))	[0, 1, 2]
values[-1]	40

Таблица 3 — состояние в графическом отладчике

Наблюдения совпали с pdb. Для списка длины 4 цикл range(3) проходит индексы 0, 1, 2; индекс 3 не достигается. Это ошибка на единицу в верхней границе диапазона. Изменять входные данные или делитель для её устранения не требуется.

6. Исправление и регрессионная проверка

Исправлена строка цикла: range(len(values) - 1) заменено на range(len(values)). Диапазон теперь включает индекс последнего элемента. Формула сложения и деление на длину списка оставлены прежними.

```
--- average.py (before)
+++ average.py (after)
@@ -6,6 +6,6 @@
     if not values:
         raise ValueError("Список не должен быть пустым")
     total = 0.0
-    for index in range(len(values) - 1):
+    for index in range(len(values)):
         total += values[index]
     return total / len(values)
```

Рисунок 6 — исправление верхней границы диапазона

До исправления создан тест test_regression_includes_last_element: среднее для [10, 20, 30, 40] должно равняться 25.0. На исходной строке этот тест обнаружил расхождение 15.0 и 25.0.

```
$ .venv/bin/python -m pytest tests/test_average.py::test_regression_includes_last_element -q
--tb=short

F                                                                        [100%]
=================================== FAILURES ===================================
_____________________ test_regression_includes_last_element _____________________
tests/test_average.py:10: in test_regression_includes_last_element
    assert average([10, 20, 30, 40]) == 25.0
E   assert 15.0 == 25.0
E    +  where 15.0 = average([10, 20, 30, 40])
=============================== short test summary info ================================
FAILED tests/test_average.py::test_regression_includes_last_element - assert 15.0 == ...

[exit code: 1]
```

Рисунок 7 — регрессионный тест падает на версии до исправления

После исправления тот же тест проходит. Исходная версия сохранена отдельно; протокол «до» воспроизводится на временном снимке.

7. Повторная проверка исправления в pdb

Исправленная версия запущена той же командой с параметром --version fixed. Точка b average.py:11 установлена перед return. Проверяются те же переменные и выражения.

```
$ .venv/bin/python -m pdb demo.py --version fixed

>
/Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/6/demo.py(1)<module>
()
-> """Одинаковый сценарий для pdb, PyCharm и обычного запуска."""
(Pdb) b average.py:11
Breakpoint 1 at
/Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/6/average.py:11
(Pdb) c
>
/Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/6/average.py(11)aver
age()
-> return total / len(values)
(Pdb) p (index, total, len(values))
(3, 100.0, 4)
(Pdb) p list(range(len(values)))
[0, 1, 2, 3]
(Pdb) p total / len(values)
25.0
(Pdb) c
Version: fixed
Values: [10, 20, 30, 40]
Average: 25.00; expected: 25.00
The program exited via sys.exit(). Exit status: 0
The program finished and will be restarted
>
/Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/6/demo.py(1)<module>
()
-> """Одинаковый сценарий для pdb, PyCharm и обычного запуска."""
(Pdb) q
```

Рисунок 8 — pdb: обработан индекс 3, сумма 100.0, среднее 25.0

Теперь (index, total, len(values)) = (3, 100.0, 4), а диапазон индексов — [0, 1, 2, 3]. После с программа выводит Average: 25.00; expected: 25.00. Pdb сообщает о нормальном sys.exit(0), предлагает повторный запуск; q завершает сессию.

8. Повторная проверка исправления в PyCharm

Конфигурация «Lab 06 - fixed» запускает demo.py с --version fixed --check. Точка на строке 11 файла average.py показывает состояние после обработки всех элементов списка.

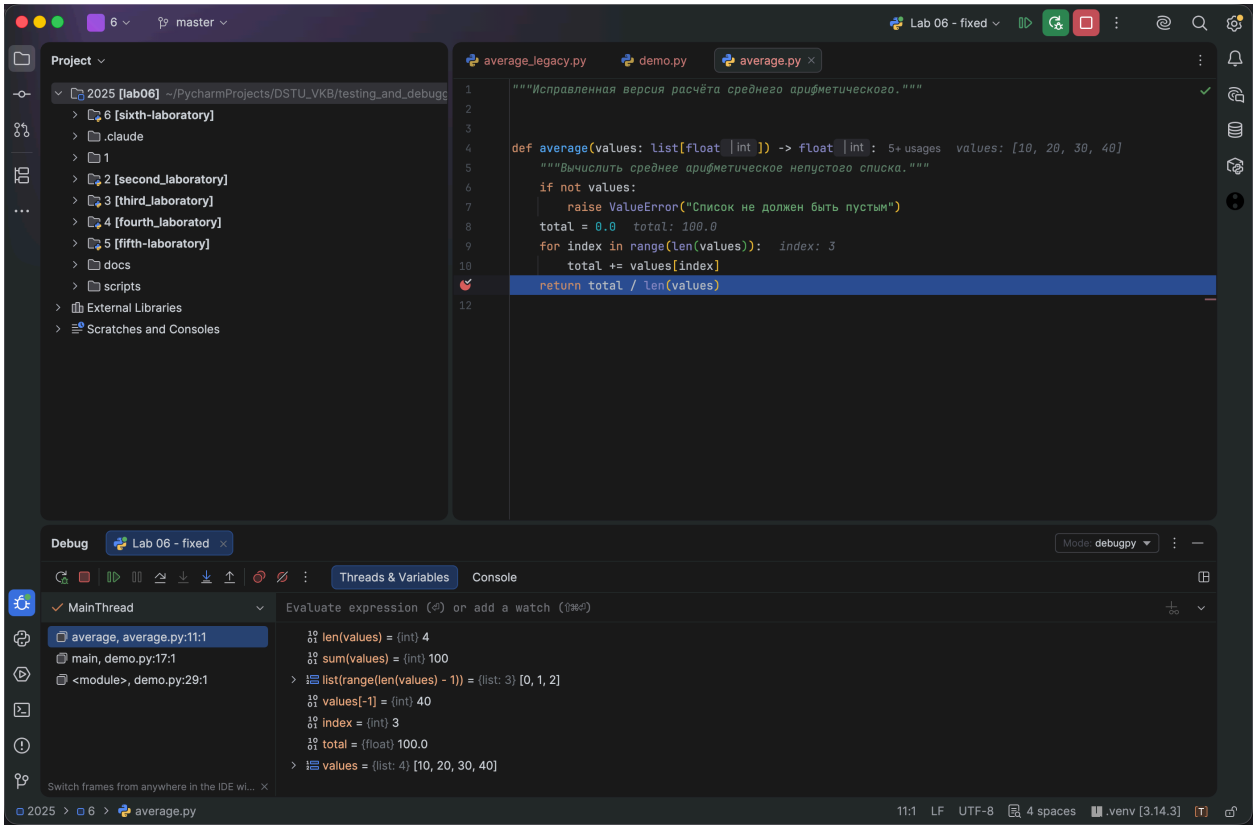


Рисунок 9 — PyCharm: исправленный цикл, index=3 и total=100.0

В Watches сохранено выражение старой границы для сравнения; выполняемый цикл на строке 9 теперь использует range(len(values)). После Resume Program консоль подтверждает правильный итог и успешное сравнение.

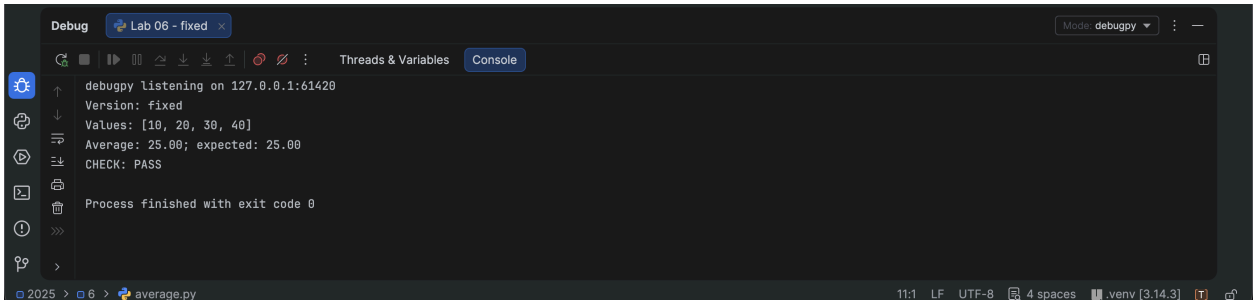


Рисунок 10 — фрагмент консоли PyCharm: Average=25.00 и CHECK: PASS

9. Итоговое тестирование и выводы

Полный набор pytest содержит 12 тестовых случаев. Проверяются регрессия на последнем элементе, воспроизводимость legacy-дефекта, список из одного значения, два значения, ненулевой последний элемент, отрицательные и дробные значения, нули, одинаковые значения, пустой список и сохранность входных данных.

```
$ .venv/bin/python -m pytest -vv --tb=short

================================ test session starts ================================
platform darwin -- Python 3.14.3, pytest-9.1.1, pluggy-1.7.0 --
/Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/6/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/6
configfile: pyproject.toml
testpaths: tests
collecting ... collected 12 items

tests/test_average.py::test_regression_includes_last_element PASSED              [  8%]
tests/test_average.py::test_legacy_reproduces_missing_last_element PASSED        [ 16%]
tests/test_average.py::test_average_cases[values0-40.0] PASSED                   [ 25%]
tests/test_average.py::test_average_cases[values1-20.0] PASSED                   [ 33%]
tests/test_average.py::test_average_cases[values2-30.0] PASSED                   [ 41%]
tests/test_average.py::test_average_cases[values3--20.0] PASSED                  [ 50%]
tests/test_average.py::test_average_cases[values4-0.0] PASSED                    [ 58%]
tests/test_average.py::test_average_cases[values5-2.0] PASSED                    [ 66%]
tests/test_average.py::test_average_cases[values6-0.0] PASSED                    [ 75%]
tests/test_average.py::test_average_cases[values7-2.0] PASSED                    [ 83%]
tests/test_average.py::test_empty_list_is_rejected PASSED                        [ 91%]
tests/test_average.py::test_input_is_not_modified PASSED                         [100%]

================================= 12 passed in 0.01s ================================

[exit code: 0]
```

Рисунок 11 — полный запуск pytest: 12 passed

Оба отладчика установили одну причину: последний элемент пропускался из-за range(len(values) - 1). Исправление восстановило сумму 100.0 и среднее 25.0. Повторные остановки перед return подтвердили правильность промежуточного состояния, а тесты — корректность результата на контрольных и граничных примерах.

Предотвращению ошибки служат тесты одиночного списка и ненулевого последнего элемента. В работе сохранена минимальная правка одной строки.

10. Сравнение инструментов и воспроизведение

Действие	pdb	PyCharm
Точка останова	b файл:строка	Щелчок по полю слева от строки
Следующая строка	n	Step Over
Продолжить	c	Resume Program
Просмотр значений	p / display	Threads & Variables / Watches
Стек вызовов	where	Панель кадров слева
Завершить	q	Stop / нормальное завершение

Таблица 4 — соответствие использованных действий

Команды из каталога лабораторной №6

```
uv sync --all-groups
.venv/bin/python demo.py --version legacy --check
.venv/bin/python -m pdb demo.py --version legacy
# Внутри pdb: b average_legacy.py:11; c; p total; q
.venv/bin/python -m pdb demo.py --version fixed
.venv/bin/python demo.py --version fixed --check
.venv/bin/python -m pytest
.venv/bin/python scripts/capture_evidence.py --stage before
.venv/bin/python scripts/capture_evidence.py --stage after
.venv/bin/python scripts/build_report.py
```

Скриншоты PyCharm сделаны в настоящем окне IDE; их нельзя получить одним запуском capture_evidence.py. Этот скрипт воспроизводит консольные сессии pdb и тесты. Иллюстрации хранятся в docs/screenshots/lab_06, отчёты — в docs/reports/lab_06; локальные протоколы report_assets исключены из Git.

Источники

1. Python Software Foundation. pdb — The Python Debugger. https://docs.python.org/3/library/pdb.html (дата обращения: 20.10.2026).

2. JetBrains. Part 1. Debugging Python Code. https://www.jetbrains.com/help/pycharm/part-1-debugging-python-code.html (дата обращения: 20.10.2026).